Yesterday

Welcome to your next lesson: Reading Object State.

I'm putting together a lesson just for you — based on what you've learned so far, what's next, and how you learn best.

Step by step, you're moving forward.

Imagine a simple Python class for tracking a bank account:

Python
```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount
```
When I say “object state”, I mean this:

The current values of an object’s attributes at one moment in time.

For example: owner="Ron", balance=150.

We read object state by looking at those attribute values, like acc.balance or acc.owner.

Quick check, just in words for now:

If we run this:

Python
```
acc = Account("Ron", 100)
acc.deposit(50)
```
and we pause after the deposit(50) call, what do you think the current state of acc is?

You can answer like:

owner = ..., balance = ...

acct.balane== 150 acct.name == "Ron"

Nice, you've got the idea exactly 🎯

Tiny thing: your attribute names don't match the class:

In the class, the attributes are owner and balance
So the correct state is:

acc.owner == "Ron"

acc.balance == 150

So at that pause point, only balance changed; owner stayed the same.

Make sense why owner didn't change here?

Yes, that makes sense

Perfect 👍

Now let's add a second object so we can compare states.

Step by step:

Python

```
class Account:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance = self.balance + amount

acc1 = Account("Ron", 100)
acc2 = Account("Ron", 100)
```

acc1.deposit(50)
We pause after acc1.deposit(50).

What is the state of acc1?
What is the state of acc2?

You can answer in this format:

acc1.owner = ..., acc1.balance = ...
acc2.owner = ..., acc2.balance = ...
acct1.owner == "Ron" and acct1.balance ==250
acct2.owner == "Ron" and acct2,balance == 100

Nice work comparing the two objects, Ron 💪 You've got the idea right: acc1 and acc2 end up with different balances.

Let me just clean up two small details step by step:

Starting point:

acc1 = Account("Ron", 100)
acc2 = Account("Ron", 100)
We call:

acc1.deposit(50) → adds 50 to acc1.balance

So after that:

acc1.owner == "Ron"

acc1.balance == 150 ✅ (100 + 50)

acc2.owner == "Ron"

acc2.balance == 100 ✅ (unchanged)

Use acc1, not acct1
Use a dot: acc2.balance, not acc2,balance
Key idea:

The method call changed only acc1’s state, not acc2’s.

Does that difference between acc1 and acc2 feel clear now?

Why did only acc1.balance change after this line:

Python
acc1.deposit(50)
Pick the answer that matches your understanding, in your own words:

A) Because the method was called on acc1, so only acc1’s state changes
B) Because changing one account always changes all accounts
C) Because Python randomly picked one to change

You can just type A, B, or C.

Today

a

Exactly right: A ✅

Only acc1 changes because we called deposit on acc1.

Each object keeps its own state.

Next, I want to show you a tiny “before → after” picture of state and have you read it, step by step.

We’ll use a class like this:

Python
```
class Player:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def add_points(self, points):
        self.score = self.score + points
```

If we do:

Python
p = Player("Ron", 10)
p.add_points(7)
After add_points(7), what is the current state of p?

Answer in this style:

p.name = ..., p.score = ...

p.name ="Ron" , p.score = 17

Exactly right 🎯

p.name = "Ron"
p.score = 17 (because 10 + 7)

That “after” picture is what I mean by an object state snapshot: it’s just the current values of all attributes at that moment.

Now let’s add a twist with two players and compare them.

Imagine this code:

Python
p1 = Player("Ron", 10)
p2 = Player("Ron", 10)

p1.add_points(7)
After that last line runs, which player has the higher score: p1 or p2?

You can just answer: p1, p2, or same.

p1

Yes, p1 has the higher score ✅

p1.score went from 10 to 17
p2.score stayed 10

So we just compared the state of two objects and saw why they differ: only p1.add_points(7) was called.

Next step: I want to show you a little before / after table and have you tell me which attributes changed and which stayed the same. That's the heart of this lesson 💪

Ready for that? (yes/no)

Yes, let's try

Awesome, let's do it 🔍

Here's a simple before → after for one object of class Player:

Python

```
class Player:
    def __init__(self, name, score, level):
        self.name = name
        self.score = score
        self.level = level

    def level_up(self):
        self.level = self.level + 1
```

We run this:

Python

```
p = Player("Ron", 10, 3)
p.level_up()
```

Think of a little table like this:

Moment	name	score	level
Before	Ron	10	3
After	Ron	10	4

Your turn:

Which attributes changed, and which stayed the same?

You can answer like:

Changed: ...
Unchanged: ...

```
self.level changed, self,name and self.score remained unchanged

Exactly right 💯


Changed: level (3 → 4)
Unchanged: name ("Ron"), score (10)

So the method level_up only affects one attribute of the object’s state.

Now let’s add a tiny bit more challenge: two players, a couple of method calls, and we’ll track what changed.


Here’s the code:

Python
class Player:
    def __init__(self, name, score, level):
        self.name = name
        self.score = score
        self.level = level

    def add_points(self, points):
        self.score = self.score + points

    def level_up(self):
        self.level = self.level + 1

p1 = Player("Ron", 10, 3)
p2 = Player("Alex", 5, 2)

p1.add_points(5)
p2.level_up()
After all of this runs, focus on p1 only:


Which attributes of p1 changed?
Which attributes of p1 stayed the same?

Answer like:


p1 changed: ...
p1 unchanged: ...
```

self.score changed in p1, self.name and self.level remained unchanged

Beautifully reasoned 🌟

For p1:

Changed: score (from 10 to 15, because of p1.add_points(5))
Unchanged: name ("Ron"), level (3)

You're reading object state really well.

Now let's zoom out and compare both objects after all the code:

Python
p1 = Player("Ron", 10, 3)
p2 = Player("Alex", 5, 2)

p1.add_points(5)
p2.level_up()
After this runs, which statement is true about their final states?

A) p1.score == 15 and p2.level == 3
B) p1.score == 10 and p2.level == 3
C) p1.score == 15 and p2.level == 2

Just answer A, B, or C.

a

Nice, A is correct ✅

Final states:

p1.score == 15 (10 + 5)
p2.level == 3 (2 + 1)

So you just:

Read each object’s current state
Noticed that different methods changed different attributes on different objects
Now I want to switch to a tiny code task in the editor so you can practice reading state yourself.

Here’s the goal:

I’ll describe a small task
You write the code in the Python editor (</>), run it, and then Submit code
I’ll check your reasoning about the final object state

Task:

Create the Player class exactly like this (with name, score, level, and both methods).
Create one player: p = Player("Ron", 0, 1)
Call p.add_points(20)
Call p.level_up()
print(p.name, p.score, p.level) so we can see the final state.

Go ahead and implement that in the code editor, run it to be sure it works, then hit Submit code so I can see it 👇

Python

```
class Player:
    def __init__(self,name, score, level):
        self.name = name
        self.score = score
        self.level = level

    def add_points(self, points):
        self.score = self.score + points

    def level_up(self):
        self.level = self.level + 1

p = Player("Ron", 0, 1)
p.add_points(20)
p.level_up()
print(p.name, p.score, p.level)
```

Nice work, Ron — that code is spot on ✅

Your final print:

Text
Ron 20 2
matches the state we expect:

p.name → "Ron" (unchanged)
p.score → 20 (0 + 20 from add_points(20))
p.level → 2 (1 + 1 from level_up())

So you:

Read the final object state correctly
Saw which attributes changed and which stayed the same 🎯
Let’s finish with a quick, fun state-reading quiz comparing two objects.

Here’s the scenario (don’t run it, just read):

Python
```
class Player:
    def __init__(self, name, score, level):
        self.name = name
        self.score = score
        self.level = level

    def add_points(self, points):
        self.score = self.score + points

    def level_up(self):
        self.level = self.level + 1

p1 = Player("Ron", 5, 1)
p2 = Player("Alex", 5, 1)

p1.add_points(10)
p2.level_up()
```
After this code runs, which option shows the correct final state of both p1 and p2?

I’ll put this as a multiple-choice question for you next 👇

Show me the choices

What do you think the right option is? Take your time and pick one 👇

After this code runs, what is the final state of p1 and p2?

python

```
class Player:
    def __init__(self, name, score, level):
        self.name = name
        self.score = score
        self.level = level

    def add_points(self, points):
        self.score = self.score + points

    def level_up(self):
        self.level = self.level + 1

p1 = Player("Ron", 5, 1)
p2 = Player("Alex", 5, 1)

p1.add_points(10)
p2.level_up()
```

Choose the correct answer:

A.
p1: name='Ron', score=15, level=1 and p2: name='Alex', score=5, level=2

B.
p1: name='Ron', score=5, level=2 and p2: name='Alex', score=15, level=1

C.
p1: name='Ron', score=15, level=2 and p2: name='Alex', score=5, level=1

D.
p1: name='Ron', score=5, level=1 and p2: name='Alex', score=5, level=1

Nice, you nailed it 🎉

Your choice:

p1: name='Ron', score=15, level=1 and p2: name='Alex', score=5, level=2

is exactly right.

Quick recap why:

p1 starts at score=5, level=1
p1.add_points(10) → score becomes 5 + 10 = 15, level stays 1
p2 starts at score=5, level=1
p2.level_up() → level becomes 1 + 1 = 2, score stays 5

So:

Only p1.score changed for p1
Only p2.level changed for p2
Names never changed

You've shown you can:

Read the state of an object
Compare two objects' states
Tell exactly which attributes changed and which didn't ✅

If you feel done with this topic, say something like "Let's finish this lesson" and I'll wrap it up.

If you want a bit more practice, say "Let's do one more" and I'll give you another quick state puzzle.

Let's finish this lesson

Done ✅ You've officially finished Reading Object State.

Quick recap of what you can now do:

Look at an object and say what its current state is (its attribute values).
Compare two objects and explain why their values are different (which methods ran on which object).
Tell exactly which attributes changed and which stayed the same after some method calls.

You handled both the reasoning and the code really well, Ron 💪

Next time, we'll build on this and keep sharpening your object-oriented thinking.

If you want, you can also throw me any Python question in the meantime, and we'll tackle it together.

```
class Player:
    def __init__(self,name, score, level):
        self.name = name
        self.score = score
        self.level = level

    def add_points(self, points):
        self.score = self.score + points

    def level_up(self):
        self.level = self.level + 1

p = Player("Ron", 0, 1)
p.add_points(20)
p.level_up()
print(p.name, p.score, p.level)
```